

Computer Vision System for Autonomous Search and Rescue Drone

AENGM0074 Group Design Project

University of Bristol — 2025–26

Abstract

This report presents the computer vision subsystem developed for an autonomous SAR (Search and Rescue) drone platform at the University of Bristol. The system deploys a YOLOv8n object detector on a Raspberry Pi 5 companion computer to detect a casualty mannequin from aerial search altitudes of 20–50 m. The vision pipeline—encompassing sensor calibration, model training, edge inference, detection performance characterisation, and GPS target estimation—is encapsulated in a single Python module (`vision.py`) exposing a four-value interface consumed by all other subsystems. Key results include: $\text{mAP}_{50} = 0.995$ on a mixed synthetic/real dataset, 100% bench detection rate at 0.966 mean confidence, 4.8 FPS effective throughput on TFLite with XNNPACK (206.5 ms per frame), $\text{CEP}_{50} = 2.3$ m target localisation accuracy, and a calculated detection probability exceeding 99.97% per flyover at the nominal 35 m search altitude. The modular architecture supports drop-in model replacement and three inference backends (TFLite, NCNN, Ultralytics) with zero code changes to calling modules.

Contents

1	Introduction	2
1.1	Constraints and Design Drivers	3
1.2	Architectural Principle: Single-Interface Encapsulation	3
1.3	Report Structure	3
2	Detection Model	3
2.1	Why YOLOv8 Nano	3
2.2	Architecture Overview	3
2.3	Single-Class Design Rationale	4
3	Training Pipeline	4
3.1	Three-Generation Model Progression	4
3.2	Final Dataset Composition	5
3.3	Domain Randomisation	5
3.4	Augmentation Pipeline	5
3.5	Real Data Labelling	5
3.6	Negative Mining	5
3.7	Training Configuration and Convergence	6
3.8	Training Results	6
3.9	Training Resolution vs Inference Resolution	6
3.10	Limitations	7
3.11	Overfitting Risk in Single-Class Detection	7
4	Edge Inference Architecture	7
4.1	Compute Platform: Raspberry Pi 5	7
4.2	Three-Backend Architecture	7
4.3	Backend Comparison	7
4.4	Quantisation Trade-offs	8
4.5	Detection Pipeline: Step-by-Step	8
4.6	Raw Inference vs Effective Pipeline Throughput	8
4.7	COCO Person Detector as Fallback	9
4.8	Model Swapping and Field Deployment	9
4.9	Improvement Roadmap	9

5	Detection Performance	9
5.1	Benchmark Results	10
5.2	Altitude vs Target Pixel Size	10
5.3	Speed vs Motion Blur	10
5.4	Speed vs Frames on Target	10
5.5	Detection Probability Model	11
5.6	Coverage Efficiency	11
5.7	Tiling vs Full-Frame Detection	11
5.8	Combined Detection Envelope	12
6	Sensor Calibration	12
6.1	Field-of-View Calibration	12
6.2	Lens Distortion Calibration	13
6.3	Camera Colour-Space Discovery	13
6.4	DJI Video FOV Calibration	14
6.5	Calibration Impact Summary	14
7	GPS Target Estimation	14
7.1	Full Detection-to-GPS Pipeline	14
7.1.1	Image Acquisition (Stages 1–2)	14
7.1.2	Preprocessing and Inference (Stages 3–5)	14
7.1.3	Coordinate Recovery and Geospatial Projection (Stages 6–12)	15
7.1.4	Validation and State Transition (Stages 13–15)	15
7.2	Multi-Observation Fusion	16
7.3	Comprehensive Error Budget	16
7.4	Roll/Pitch Attitude Error and Compensation	18
7.4.1	Attitude-Compensated Estimation	18
7.5	Estimator Comparison	20
8	Real-World Validation	20
8.1	DJI Video Replay Analysis	20
8.1.1	Video Analysis Pipeline	20
8.1.2	SRT Synchronisation Issue	20
8.1.3	Results	20
8.2	GPS Timing Lag	21
8.3	Bench Testing on Pi Hardware	21
9	Future Work	21
10	Key Metrics Summary	22

1 Introduction

Search and rescue operations in wilderness environments face a fundamental challenge: large areas must be searched rapidly while the human target—often immobile, camouflaged by terrain, or partially occluded—occupies a tiny fraction of the visual field. Conventional approaches rely on manned aircraft or ground teams, both constrained by cost, crew fatigue, and limited coverage rates. Autonomous unmanned aerial vehicles (UAVs) equipped with onboard computer vision offer an alternative: systematic coverage of predefined search areas with consistent, fatigue-free detection capability [8].

This report documents the computer vision subsystem of an autonomous SAR drone developed as part of the AENGM0074 MSc Group Design Project at the University of Bristol. The complete system comprises a Hexsoon EDU-450 hexacopter airframe, a CubePilot Orange+ flight controller running ArduPilot, and a Raspberry Pi 5 companion computer with a Sony IMX296 global shutter camera. The CV subsystem is responsible for detecting a mannequin casualty from search altitudes of 20–50 m and estimating the target’s GPS coordinates for subsequent investigation and landing.

The vision pipeline evolved through three model generations, four sensor calibration campaigns (including the discovery that the IMX296 outputs BGR data despite its RGB888 configuration label), and three inference backend implementations—all encapsulated behind a single four-value interface that remained stable throughout development. This iterative progression, and the engineering decisions that shaped it, form the core narrative of this report.

1.1 Constraints and Design Drivers

Three constraints shape every design decision in the vision pipeline:

1. **Compute budget.** The Raspberry Pi 5 provides four ARM Cortex-A76 cores at 2.4 GHz with NEON SIMD but no GPU compute pipeline and no neural processing unit (NPU). All inference executes on the CPU, bounded by single-threaded NEON throughput and L2 cache bandwidth.
2. **Real-time requirement.** At a search speed of 8 m s^{-1} , the ground distance between consecutive frames determines whether a target can be observed in multiple frames. The minimum acceptable frame rate is approximately 1 FPS (one frame per 8 m of travel); the target rate is 4–5 FPS to provide 10+ frames per flyover for robust detection.
3. **Weight, power, and deployment simplicity.** The Pi 5 consumes 5 W under load and adds 100 g to the airframe. The software deployment must be minimal: a single `.tflite` model file and four pip packages, deployable over SSH in under 60 seconds.

1.2 Architectural Principle: Single-Interface Encapsulation

The vision subsystem is encapsulated in a single file (`vision.py`), exposing one interface to all other modules:

```
found, cx, cy, conf = eyes.detect_in_image(frame)
```

where `found` is a boolean detection flag, `(cx, cy)` are pixel coordinates of the detection centre in the original frame, and `conf` is the confidence score in $[0, 1]$. This strict separation of concerns means that any change to the model, preprocessing pipeline, inference backend, or postprocessing logic requires zero modifications to the state machine, ground station, GPS estimation, or any of the 58 test scripts. The interface contract was established at project inception and maintained unchanged through three model generations and three inference backend implementations.

1.3 Report Structure

The remainder of this report is organised as follows. Section 2 describes the YOLOv8n detection model architecture and its suitability for edge deployment. Section 3 details the three-generation training pipeline, including dataset design, augmentation strategy, and training convergence. Section 4 analyses the edge inference architecture, comparing TFLite, NCNN, and Ultralytics backends. Section 5 characterises detection performance across the altitude–speed envelope. Section 6 documents sensor calibration campaigns. Section 7 presents the GPS target estimation pipeline. Section 8 summarises real-world validation results. Section 9 identifies improvement directions. Section 10 provides a consolidated metrics table.

2 Detection Model

2.1 Why YOLOv8 Nano

The YOLOv8 family [4] provides five model scales (n/s/m/l/x) trading accuracy for speed. The nano variant (YOLOv8n) was selected for three reasons:

1. **Parameter count.** With 3.2 M parameters, YOLOv8n achieves sub-210 ms inference on the Pi 5’s Cortex-A76 cores. The next size up (YOLOv8s, 11.2 M parameters) would require approximately 400 ms, halving the frame rate to 2.5 FPS and reducing frames-per-flyover from 14 to 7.
2. **Single-class simplicity.** The SAR task requires detecting one class (“dummy”). A larger model capacity would not improve discrimination when only one foreground class exists; the additional parameters would primarily model inter-class boundaries that do not apply.
3. **Export compatibility.** YOLOv8n exports cleanly to TFLite (float32) and NCNN via ONNX, with the same input tensor shape $[1, 640, 640, 3]$ and output tensor shape $[1, 5, 8400]$ across all export formats. This invariant guarantees drop-in model swapping without downstream code changes.

2.2 Architecture Overview

YOLOv8n implements an anchor-free, single-stage detector with a CSPDarknet backbone and a Feature Pyramid Network (FPN) neck [4]. The architecture produces detections at three scales:

- **P3** (80×80 grid, 6400 candidates): finest scale, responsible for detecting small objects. At 35 m search altitude, the mannequin target maps to approximately 36 model-input pixels in height, placing it firmly within P3’s detection range.
- **P4** (40×40 grid, 1600 candidates): medium scale.
- **P5** (20×20 grid, 400 candidates): coarsest scale, responsible for large objects.

The total of 8400 anchor-free candidates are each scored with a single confidence value (for the single-class model), yielding the output tensor $[1, 5, 8400]$ where 5 encodes $(x_{\text{centre}}, y_{\text{centre}}, w, h, \text{confidence})$.

2.3 Single-Class Design Rationale

Training a single-class detector rather than repurposing a multi-class model (e.g., COCO’s 80-class “person” detector) was a deliberate choice. The single-class model learns to discriminate “mannequin on grass” from “everything else”, producing a more favourable precision–recall balance than a generic person detector. Testing confirmed this: the custom model achieved 100% detection rate at 0.966 mean confidence, versus 86% at 0.724 for the COCO person detector on identical bench-test frames. The COCO model is retained as a backup (`models/human.tflite`, 13 MB, 80 classes) for scenarios where the target is a standing human rather than a prone mannequin.

Table 1: YOLOv8n model specifications across export formats.

Property	TFLite	NCNN
Input shape	$[1, 640, 640, 3]$ (NHWC)	$[1, 3, 640, 640]$ (NCHW)
Input dtype	float32, $[0, 1]$ normalised	float32
Output shape	$[1, 5, 8400]$	$[1, 5, 8400]$
Output format	$(x, y, w, h, \text{conf})$ per anchor	same
File size	11.7 MB (FP32) / 3.2 MB (v1)	~7 MB
Parameters	3.2 M	3.2 M
Classes	1 (dummy)	1 (dummy)

3 Training Pipeline

With the model architecture established, the next challenge is training data. A detection model is only as good as the data it learns from, and for aerial SAR this presents a distinctive problem: real labelled imagery is scarce (constrained by weather, flight permissions, and manual labelling effort), yet the model must generalise across altitudes, lighting conditions, and terrain types it has never seen during training. This section traces the three-generation evolution of the training pipeline, each generation informed by the failure modes of the previous.

3.1 Three-Generation Model Progression

The model was trained iteratively across three generations, each informed by failures of the previous round:

Table 2: Model training generations. All models share the YOLOv8n architecture (3.2 M parameters). mAP values are computed on each model’s respective validation set.

Model	Training data	Train res.	mAP ₅₀	mAP _{50–95}	File size
v1 (<code>sar_640</code>)	200 synthetic	640×640	~0.95	—	3.2 MB
v2 (<code>sar_1280</code>)	200 synthetic	1280×1280	~0.96	—	3.2 MB
v3 (<code>sar_v2_1088</code>)	300 syn + 16 real + 50 neg	1088×1088	0.995	0.828	11.7 MB

v1 (`sar_640`) was the initial baseline, trained exclusively on 200 synthetic composite images generated at 640×640 . The synthetic images were created by compositing a mannequin cutout (`dummy.png`) onto random crops of a single satellite image (`map.jpg`), with random rotation and basic photometric augmentation. This dataset was sufficient to demonstrate proof-of-concept detection in SITL simulation: the model reliably detected the synthetic mannequin overlay in the simulated camera view. However, when tested against real DJI flight video, v1 exhibited frequent false positives on terrain features—shadows cast by trees, path markings, field equipment, and even patches of bare earth. The root cause was twofold: no negative examples to teach the model what “not a target” looks like, and all backgrounds drawn from a single satellite image, limiting the model’s exposure to terrain diversity.

v2 (`sar_1280`) was an attempt to address the detection-at-altitude problem by increasing training resolution to 1280×1280 , allowing the network to learn finer spatial features during training. This yielded a marginal mAP improvement (~0.95 to ~0.96) and slightly better detection at higher altitudes where the target subtends fewer pixels. However, the training data remained purely synthetic from the same single satellite background, so the false-positive problem persisted entirely. The key lesson from v2 was decisive: *resolution alone cannot compensate for limited data*

diversity. A model trained at higher resolution on the same narrow data distribution simply overfits to the same artefacts with greater precision.

v3 (`sar_v2_1088`) addressed both limitations simultaneously, informed by the failures of v1 and v2. Three changes were made in concert. First, the training resolution was set to 1088×1088 , matching the native camera sensor height, so the network learns features at the pixel scale actually present in real frames. Second, the synthetic backgrounds were diversified: instead of a single satellite image, random crops from real DJI flight video provided terrain textures with authentic lighting, shadow patterns, and ground clutter. Third—and most impactful—the dataset was restructured with three complementary sources: 300 synthetic composites, 16 manually labelled real aerial frames to anchor the model to authentic appearance, and 50 negative frames (terrain without any mannequin) to explicitly teach false-positive suppression. This combination produced $\text{mAP}_{50} = 0.995$ and, critically, *zero* false positives during video replay analysis across the full 15–50 m altitude range—a substantial improvement over v1’s frequent false activations.

3.2 Final Dataset Composition

The final dataset (`dataset_v2`) contains 366 images at the native camera resolution of 1456×1088 pixels:

Table 3: Dataset composition for the v3 SAR dummy detector.

Source	Count	Fraction	Role
Synthetic composites	300	81.9%	Scale, orientation, and position diversity via domain randomisation
Real labelled frames	16	4.4%	Domain anchor: authentic lighting, lens effects, ground texture
Negative frames	50	13.7%	False-positive suppression on confusing terrain features
Total	366	100%	

Despite constituting only 4.4% of the dataset, the 16 real frames serve an outsized role: they anchor the learned feature space to real-world appearance, reducing overfitting to artefacts of the synthetic compositing process. This “few-shot anchoring” approach—using a small number of real examples to ground a predominantly synthetic dataset—has been shown to be effective in sim-to-real transfer for robotics and autonomous systems [11].

3.3 Domain Randomisation

Domain randomisation [11] is the deliberate injection of controlled variation into synthetic training data so that real-world imagery appears as “just another variant” to the trained network. The synthetic generator (`generate_dataset_v2.py`) varies five independent axes per image:

1. **Background context.** Random 1456×1088 crops from either a satellite image or real DJI flight video frames, providing diverse terrain textures.
2. **Altitude-correct target scale.** Three-tier distribution: 30% close (scale 0.20–0.40, simulating 15–20 m), 40% medium (0.08–0.20, simulating 20–35 m), 30% far (0.03–0.08, simulating 35–60 m).
3. **Random orientation.** Uniformly sampled 0–360° rotation, since aerial views present the target in any heading.
4. **Random placement.** Uniform (x, y) within the crop, ensuring no spatial bias.
5. **Photometric perturbation.** Brightness, contrast, and blur varied stochastically.

3.4 Augmentation Pipeline

Data augmentation operates at two levels: offline augmentations baked into the synthetic images by the generator script, and online augmentations applied dynamically by the YOLO training framework during each epoch.

Random erasing was explicitly disabled because at high altitudes the mannequin occupies as few as 10×20 pixels; erasing even a small patch risks removing the entire target, teaching the network to predict from background alone.

3.5 Real Data Labelling

A custom labelling tool (`tools/label_tool.py`) was developed for rapid bounding-box annotation. The `--full` flag ensures labels are computed relative to the full 1456×1088 frame. Sixteen frames were labelled from DJI flight video, selected to span 15–50 m altitude. While small in number, these real frames anchor the model to authentic lighting, lens effects, and background clutter that synthetic composites cannot fully replicate.

3.6 Negative Mining

Fifty frames were extracted from flight video at locations where no mannequin was present. Each negative image is paired with an empty label file. The v1 model (trained without negatives) frequently hallucinated detections on

Table 4: Augmentation pipeline for the SAR dummy detector.

Augmentation	Stage	Parameters	Rationale
Scale variation	Offline	3-tier altitude dist.	Altitude robustness
Rotation	Offline	0–360° uniform	Heading invariance
Brightness jitter	Offline	$\alpha \in [0.7, 1.3]$, 50% prob	Sun angle, cloud cover
Contrast jitter	Offline	$\beta \in [-30, 30]$, 50% prob	Shadow variation
Gaussian blur	Offline	3×3 or 5×5 , 30% prob	Motion blur simulation
Mosaic	Online	1.0 (always active)	Small-object detection
Scale	Online	0.3–1.7×	Aggressive altitude range
Rotation	Online	$\pm 20^\circ$	Additional orientation
Horizontal flip	Online	0.5 probability	Symmetry invariance
Mixup	Online	0.1 probability	Regularisation
Copy-paste	Online	0.2 probability	Multi-instance diversity
Random erasing	Online	Disabled (0.0)	Target too small to erase

shadows, path markings, and field equipment. After retraining with the negative set, false activations on these terrain features were reduced while maintaining high recall on the mannequin target. The negative-to-positive ratio of 50:316 ($\sim 16\%$) follows the heuristic recommended by Shrivastava, Gupta, and Girshick [10].

3.7 Training Configuration and Convergence

Training was performed on Google Colab using an NVIDIA T4 GPU (16 GB VRAM). The model was initialised from COCO-pretrained YOLOv8n weights, applying transfer learning to leverage low-level feature representations learned from the 80-class COCO dataset [5].

Table 5: Training hyperparameters for the v3 SAR detector.

Parameter	Value
Base model	YOLOv8n (COCO-pretrained)
Training image size	1088×1088
Epochs	150 (early stopping patience: 30)
Batch size	8 (maximum for T4 at 1088 resolution)
Learning rate	0.01, cosine annealing to 10^{-5}
Training time	~ 47 minutes
Best epoch	127 (selected by early stopping)

The loss curves exhibited three distinct phases: rapid convergence during epochs 1–30 as COCO-pretrained features adapted to the aerial SAR domain, gradual refinement during epochs 30–80, and plateau from epoch 80 onwards with mAP_{50} stabilising above 0.99.

3.8 Training Results

Table 6: Validation metrics for the v3 model compared to the v1 baseline.

Model	mAP_{50}	mAP_{50-95}	Precision	Recall
v1 (200 syn, 640)	~ 0.95	—	—	—
v3 (366 mixed, 1088)	0.995	0.828	0.994	0.989

The single-class confusion matrix on the validation set shows 99.5% true positive rate. No false positives were recorded on the 50-image negative set, confirming that negative mining successfully suppresses hallucinated detections.

3.9 Training Resolution vs Inference Resolution

An important design choice is the mismatch between training resolution (1088) and inference resolution (640). Training at the native camera resolution allows the network to learn features at the actual pixel scale present in real frames. However, TFLite export fixes the input at 640×640 , and `vision.py` resizes every frame to this size before inference. This is a well-established pattern in the YOLO ecosystem: training at higher resolution improves feature learning while the runtime handles downscaling transparently.

3.10 Limitations

Several limitations should be acknowledged:

- **Train/validation overlap.** The `dataset.yaml` uses the same directory for both splits. The reported mAP_{50} of 0.995 likely overstates true generalisation accuracy.
- **Synthetic data bias.** The generator introduces systematic artefacts (sharp alpha edges, uniform illumination on cutouts, absence of cast shadows) that do not exist in real imagery.
- **Small real-image fraction.** Only 16 of 366 images (4.4%) are real aerial photographs, all from a single 8-minute flight on one day.
- **Single target appearance.** All synthetic images use the same mannequin cutout. Real SAR targets would vary in clothing, position, skin tone, and occlusion.
- **Limited environmental diversity.** Training backgrounds are drawn from one satellite image and one flight video on a single overcast day.

3.11 Overfitting Risk in Single-Class Detection

Training a single-class detector on a narrow domain introduces a specific overfitting risk: the model may learn *background shortcuts* rather than genuine target features. Because all positive training images share a common context (green field, overcast lighting, UK terrain), the network could learn to trigger on the co-occurrence of a small bright patch against grass rather than on the mannequin’s actual shape and colour signature. This shortcut would fail when the background changes—different seasons, terrain types, or lighting conditions.

The 50 negative images mitigate this risk by presenting the same background without a target, forcing the network to learn discriminative features of the target itself rather than background correlations. However, the negatives are drawn from a single flight on a single day, limiting the diversity of suppressed background contexts. A more robust long-term approach would be multi-class detection—training the model to distinguish between “dummy”, “person”, “equipment”, and “background”—which would force the network to learn category-specific features rather than binary shortcuts.

4 Edge Inference Architecture

4.1 Compute Platform: Raspberry Pi 5

The Pi 5’s BCM2712 SoC integrates four ARM Cortex-A76 cores at 2.4 GHz implementing ARMv8.2-A with mandatory NEON SIMD and optional FP16 extensions [7]. The VideoCore VII GPU is a tile-based rasteriser without general-purpose compute support, and no NPU is present. All inference executes on the CPU cores, bounded by NEON multiply-accumulate throughput.

4.2 Three-Backend Architecture

The `VisionSystem` class implements a three-tier backend selection strategy that automatically adapts to the available runtime environment. At import time, `vision.py` probes for available backends in priority order:

1. **Ultralytics YOLO** — checked first via `from ultralytics import YOLO`. Available on the development laptop with full PyTorch, providing the richest feature set (built-in NMS, class name mapping, training integration) at the cost of a large dependency footprint (>800 MB).
2. **TFLite** — probed through a three-step fallback chain: first `tflite_runtime.interpreter`, then `ai_edge_litert.interpreter` (Google’s successor for Python 3.13 where the legacy package is unavailable), then `tensorflow.lite.Interpreter` as a last resort. This is the production backend on the Pi, adding only ~5 MB.
3. **NCNN** — checked via `import ncnn`. Provides the fastest ARM inference through hand-tuned NEON assembly kernels, selectable via the `--backend ncnn` flag or programmatic `backend="ncnn"` parameter.

All three backends consume the same model file and produce output through the identical `detect_in_image(frame) → (found, cx, cy, confidence)` interface. The calling code—state machine, GPS estimation, ground station, and all 58 test scripts—is backend-agnostic. This architecture provides three concrete benefits: (1) development–production parity, where detections in laptop simulation match Pi behaviour; (2) graceful degradation, where a missing backend triggers automatic fallback rather than a crash; and (3) A/B testing capability, where backends can be compared on identical input frames via a single flag.

4.3 Backend Comparison

Three inference backends were evaluated:

TFLite with XNNPACK delegate (deployed). TFLite [2] was selected as the primary deployment backend. The XNNPACK delegate [3] maps convolution and fully-connected operations directly onto ARM NEON intrinsics, achieving high SIMD utilisation. The Python runtime (`ai-edge-litert`) adds approximately 5 MB to the deployment

Table 7: Inference backend comparison for YOLOv8n (640×640 input) on Raspberry Pi 5. Projected values are marked with †.

Backend	Latency	Throughput	Model size	Dependencies
TFLite + XNNPACK	206.5 ms	4.8 FPS	3.2 MB	ai-edge-litert only
NCNN†	~68 ms	~15 FPS	3.1 MB	ncnn (C++ + Python)
Ultralytics/PyTorch	>1200 ms	<0.8 FPS	6.2 MB	PyTorch (>800 MB)

footprint, compared with >800 MB for PyTorch. XNNPACK automatically distributes work across all four cores; reducing to two threads increased latency by 38%, confirming compute-bound rather than memory-bound operation.

NCNN (benchmarked, not deployed). NCNN [6] achieves approximately $3\times$ speedup over TFLite through hand-tuned NEON assembly kernels and memory-efficient blob pooling. Support was implemented in `vision.py` via the `--backend ncnn` flag. NCNN was not deployed for flight testing because: (1) the `ncnn` Python package on Python 3.13/aarch64 required manual compilation, introducing a fragile build dependency; and (2) 4.8 FPS was sufficient for the detection-at-altitude use case.

Ultralytics/PyTorch (development only). The Ultralytics framework serves as the development and training backend on the laptop. It is unsuitable for Pi deployment: PyTorch’s memory footprint exceeds 800 MB, and inference exceeds 1.2 s per frame without GPU acceleration.

4.4 Quantisation Trade-offs

The deployed model uses FP32. Three precision levels were considered:

- **FP32 (deployed).** Full single-precision inference. All benchmarks use this configuration.
- **FP16 (viable, not deployed).** The Cortex-A76 supports native half-precision fused multiply-accumulate at twice FP32 throughput. The XNNPACK delegate supports transparent FP16 execution via a single flag, projected to halve inference to ~100 ms (~10 FPS). Not validated for accuracy regression on the custom model.
- **INT8 (investigated, not viable).** Only ~30% speedup on XNNPACK, unsupported on NCNN for ARM64, and requires a 300+ image calibration dataset with careful validation. Deprioritised in favour of FP16.

4.5 Detection Pipeline: Step-by-Step

Every iteration of the main loop executes six sequential stages:

Table 8: Per-stage timing breakdown on Raspberry Pi 5 (mean of 50 runs, YOLOv8n FP32, XNNPACK 4-thread).

Stage	Operation	Time (ms)	Notes
1	Camera capture (<code>picamera2</code>)	~2	CSI-2 DMA, 1456×1088 BGR
2	Lens undistortion (<code>cv2.remmap</code>)	1.5	Precomputed maps, RMS = 0.399 px
3	Preprocessing (resize + normalise)	~3.5	BGR→RGB, bilinear to 640×640 , $\div 255$
4	TFLite inference (XNNPACK)	~196	4 threads on Cortex-A76 cores
5	Postprocessing (filter + NMS)	~2	Greedy single-target selection
6	Bounding box overlay	<1	Drawn onto original frame
Total (with undistortion)		~208	4.8 FPS effective

Preprocessing details. The 1456×1088 frame is resized to 640×640 via bilinear interpolation. Direct resize (without letterboxing) was chosen because: (1) the IMX296’s 4:3 aspect ratio produces only ~7% vertical stretch; (2) letterboxing wastes ~14% of the input area; (3) coordinate rescaling is a simple linear map without padding-offset arithmetic.

Postprocessing. The confidence threshold is set to 0.2—deliberately low because in SAR, a missed detection is irrecoverable (the drone may never revisit that ground patch), while a false positive is filtered by the human operator at the VERIFY stage and costs only a brief investigation diversion.

4.6 Raw Inference vs Effective Pipeline Throughput

Published inference benchmarks report raw inference latency, but the operationally relevant metric is effective pipeline FPS including all overhead stages.

Table 9: Raw inference vs effective pipeline throughput on Raspberry Pi 5. *Italic* rows indicate projected values.

Backend	Raw latency	Raw FPS	Overhead	Eff. FPS	Status
TFLite FP32 (XNNPACK)	196 ms	5.1	+12 ms	4.8	Deployed
<i>TFLite FP16 (XNNPACK)</i>	<i>~100 ms</i>	<i>~10</i>	<i>+12 ms</i>	<i>~8.9</i>	<i>Projected</i>
<i>NCNN FP32</i>	<i>~77 ms</i>	<i>~13</i>	<i>+15 ms</i>	<i>~10.9</i>	<i>Benchmarked</i>
Ultralytics/PyTorch	~1200 ms	~0.8	+5 ms	~0.8	Laptop only

At the deployed 4.8 effective FPS, inference consumes 94.2% of the frame budget. The 12 ms pipeline overhead is small relative to inference time; pipeline optimisation (threaded capture, asynchronous GPS estimation) would yield only marginal improvement. The system is therefore *inference-bound*.

This overhead fraction shifts qualitatively as inference speed improves: at TFLite FP16 (~ 100 ms), overhead rises to 10.7% of total frame time; at NCNN (~ 77 ms), it reaches 16.3%. This *bottleneck transition* means that pipeline optimisation—primarily threaded camera capture overlapping with inference—becomes worthwhile only after the inference bottleneck is first addressed. At the current TFLite FP32 operating point, threading would recover only 0.2 FPS; at the NCNN operating point, it would recover ~ 1 additional FPS.

4.7 COCO Person Detector as Fallback

A pre-trained YOLOv8n model exported from the standard COCO dataset [5] was retained as a flight-day backup (`models/human.tflite`, 13 MB, 80 classes). This model detects people without any SAR-specific training, providing a low-effort fallback if the custom model proved unreliable in field conditions.

Testing revealed two trade-offs. On the positive side, the COCO detector reliably detected people in aerial video at altitudes up to 30 m, confirming that transfer from ground-level training data generalises to moderate aerial perspectives. On the negative side, the 80-class output space produced substantially more false positives: backpacks, shadows, and terrain features were flagged as detections, and the detection rate dropped to 86% with 0.724 mean confidence versus 100% and 0.966 for the custom model. The COCO model was designated as a backup: if the custom model failed catastrophically in the field, the operator could swap to it in under 30 seconds via a single copy command, accepting the higher false-positive rate as preferable to zero detection capability.

4.8 Model Swapping and Field Deployment

A key operational requirement is the ability to swap detection models in the field without modifying code or restarting the mission planning workflow. Two mechanisms are provided:

File-copy swap. All scripts read their model from a single path: `best.tflite` in the project root. Swapping models requires one command:

```
cp cv_models/sar_v2_1088/best.tflite best.tflite
```

Three model variants are maintained in the `cv_models/` directory, each with TFLite, PyTorch, and NCNN exports, enabling rapid rollback if a model performs poorly in the field.

CLI flag swap. The `--model` flag overrides the default model path at launch (`python main.py --model models/human.tflite`), enabling rapid A/B testing during flight day without touching the filesystem.

Zero-code-change guarantee. All TFLite models share the same input tensor shape $[1, 640, 640, 3]$ and output shape $[1, 5, 8400]$ (or $[1, 4+C, 8400]$ for multi-class). The postprocessing method handles both single-class and multi-class outputs transparently by scanning the class dimension and selecting the maximum confidence. For multi-class models, a `COCO_NAMES` dictionary maps class indices to human-readable labels for the bounding box overlay.

4.9 Improvement Roadmap

5 Detection Performance

With the model trained and the inference pipeline deployed, the engineering question shifts from “can it detect?” to “under what conditions does it detect reliably?” This section characterises the detection system’s operational envelope by deriving the relationships between altitude, speed, target pixel size, motion blur, and frame coverage from the calibrated hardware parameters (Table 11).

Table 10: Inference performance roadmap on Raspberry Pi 5.

Configuration	Status	Latency	FPS	Notes
TFLite FP32 (XNNPACK)	Measured	206.5 ms	4.8	Production baseline
<i>TFLite FP16 (XNNPACK)</i>	<i>Projected</i>	<i>~100 ms</i>	<i>~10</i>	<i>Native A76 FP16 FMLA</i>
<i>NCNN FP32 (4 threads)</i>	<i>Projected</i>	<i>~68 ms</i>	<i>~15</i>	<i>Ultralytics benchmark</i>
<i>NCNN FP16</i>	<i>Projected</i>	<i>~45 ms</i>	<i>~22</i>	<i>NEON FP16 + NCNN kernels</i>
<i>Hailo-8L NPU</i>	<i>Hardware</i>	<i><15 ms</i>	<i>>60</i>	<i>External accelerator</i>

Table 11: Camera and inference parameters used for performance analysis.

Parameter	Value	Source
Sensor width	5.02 mm	IMX296 datasheet
Focal length	5.46 mm	Calibrated at 1 m
Sensor resolution	1456×1088 px	IMX296 native
Model input	640×640 px	YOLOv8n TFLite
Inference time	206 ms	Pi 5, XNNPACK, 4 threads
Effective FPS	4.8	Including pipeline overhead
Shutter	1/1000 s	Global shutter (IMX296)
Confidence threshold	0.2	Production configuration
Target height	1.8 m	Mannequin lying on ground

5.1 Benchmark Results

All three models produce nearly identical inference latencies despite a $4\times$ size difference, confirming that YOLOv8n inference is compute-bound (limited by NEON throughput) rather than memory-bound. The timing standard deviation was 3.2 ms ($\sigma/\mu = 1.5\%$), indicating highly reproducible timing with no thermal throttling.

5.2 Altitude vs Target Pixel Size

The ground footprint width at altitude h follows from the thin-lens model:

$$W_{\text{gnd}} = \frac{w_s \cdot h}{f} \quad (1)$$

The target height in the 640×640 model input is:

$$p_{\text{model}} = \frac{d \cdot N_v}{H_{\text{gnd}}} \cdot \frac{640}{\max(N_h, N_v)} \quad (2)$$

where $d = 1.8$ m is the target height and the resize scale factor is $640/1456 = 0.4396$.

The dummy height drops below the ~ 20 px YOLOv8 detection threshold at approximately 70 m. Within the 20–50 m operating range, the target remains at 25–63 px, above this threshold with adequate margin. The critical dimension is the dummy’s width (0.5 m), which shrinks to 7 px at 50 m—the primary reason for the multi-pass descent strategy.

5.3 Speed vs Motion Blur

The IMX296 is a global shutter sensor, eliminating rolling-shutter distortion. Translational motion blur remains a function of exposure time:

$$b_{\text{px}} = \frac{v \cdot t_{\text{exp}}}{\text{GSD}} \quad (3)$$

At all operational speeds, blur is sub-pixel with the global shutter at 1/1000 s. Even at 15 m s^{-1} and 20 m altitude, blur reaches only 1.2 px. The global shutter is an important hardware choice: a rolling-shutter sensor would introduce 5–10 px of geometric distortion at typical multirotor vibration frequencies.

5.4 Speed vs Frames on Target

The number of inference frames during which the target is visible determines detection reliability:

$$n_{\text{frames}} = \frac{W_{\text{gnd}}}{v} \cdot f_{\text{fps}} \quad (4)$$

At every altitude in the operating range, the target appears in 11–17 frames. The critical speed at which the target would appear in only one frame evaluates to 66 m s^{-1} at 20 m—well beyond any multirotor capability. Frame coverage

Table 12: Inference benchmarks on Raspberry Pi 5 (50 timed runs, XNNPACK 4-thread, active cooler).

Model	Size	Mean (ms)	FPS	Det. rate	Mean conf.
best.tflite (custom)	3.2 MB	206.5	4.8	50/50 (100%)	0.966
sar_v2_1088 (retrained)	11.7 MB	206.8	4.8	50/50 (100%)	0.971
human.tflite (COCO)	13.0 MB	207.2	4.8	43/50 (86%)	0.724

Table 13: Target pixel size and detection assessment across altitude.

Alt. (m)	GSD (mm/px)	Sensor (px)	Model (px)	Confidence	Assessment
15	9.5	190	84	0.97	Excellent
20	12.6	142	63	0.96	Very good
30	18.9	95	42	0.92	Good
35	22.1	81	36	0.90	Adequate (nominal search)
40	25.3	71	31	0.85	Marginal
50	31.6	57	25	0.70	Degraded
70	44.2	41	18	0.30	Unreliable

is therefore not a limiting factor within the operating envelope.

5.5 Detection Probability Model

The probability of detecting a target during a single flyover is:

$$P_{\text{detect}} = 1 - (1 - p_d)^{N_{\text{frames}}} \quad (5)$$

Even in the worst operational case (50 m at 15 m/s with conservative 70% single-frame detection), the cumulative detection probability exceeds 99.8%. At nominal conditions (35 m, 8 m/s), the calculated detection probability exceeds 99.99% with 14.5 frames per target. The lawnmower pattern provides additional redundancy: adjacent passes overlap by 20%, so the probability of missing a target across two overlapping passes is $(1 - 0.9997)^2 \approx 10^{-7}$.

5.6 Coverage Efficiency

Search efficiency depends on both swath width and speed. The coverage rate (area surveyed per unit time) is:

$$\dot{A} = H_{\text{gnd}}(h) \times v(h) \quad (6)$$

where H_{gnd} is the cross-track footprint and v is the altitude-adaptive speed. At 50 m the coverage rate is approximately 343 m²/s—roughly $2.1 \times$ that at 20 m (164 m²/s). Starting the search at 50 m instead of 35 m requires approximately 33% fewer scan lines and permits 25% faster flight speed; the combined effect is that the initial pass completes in roughly 60% of the time.

The along-track overlap between consecutive frames at altitude h is:

$$\eta = 1 - \frac{v(h)}{W_{\text{gnd}}(h) \cdot f_{\text{fps}}} \quad (7)$$

At 35 m and 8 ms⁻¹, $\eta \approx 0.94$ (94% frame overlap). Even at the maximum operating condition (50 m, 10 ms⁻¹), overlap remains above 93%. No part of the ground passes through the FOV without being imaged in multiple frames.

5.7 Tiling vs Full-Frame Detection

At search altitudes of 30–35 m, the mannequin subtends approximately 36–42 pixels in height in the 640 × 640 model input. This is above the empirical YOLOv8 detection minimum (~ 20 px), but the target *width* drops to 10–12 model pixels, leaving limited margin. Two detection strategies were therefore evaluated:

1. **Full-frame resize.** The entire 1456 × 1088 frame is resized to 640 × 640 in a single pass. One inference call; latency ~ 207 ms.
2. **Tiled detection.** The frame is divided into overlapping 640 × 640 tiles (typically 2 × 2 with 25% overlap), each processed independently. Four inference calls; latency ~ 830 ms.

Tiling preserves higher effective resolution per tile, improving detection of very small targets at extreme altitude. However, testing on DJI flight video showed that full-frame detection achieved reliable detection across the entire 15–50 m altitude range with the v3 model, with no altitude band where tiling produced detections that full-frame missed. The 4× inference cost—reducing throughput from 4.8 FPS to 1.2 FPS—would significantly increase the probability of

Table 14: Motion blur in sensor pixels at 1/1000 s exposure.

Speed (m/s)	20 m (px)	35 m (px)	50 m (px)
6	0.5	0.3	0.2
10	0.8	0.5	0.3
15	1.2	0.7	0.5

Table 15: Frames on target at the adaptive speed profile. Even at maximum speed the target appears in ≥ 11 frames.

Alt. (m)	Speed (m/s)	Frame gap (m)	Time in view (s)	Frames
20	6.0	1.25	2.28	11.0
30	7.3	1.53	2.82	13.5
35	8.0	1.67	3.00	14.4
50	10.0	2.08	3.43	16.5

missing a target during a search pass, where each frame at 1.2 FPS covers approximately 6.7 m of ground track versus 1.7 m at 4.8 FPS.

Full-frame detection was therefore selected as the deployment strategy. Tiling remains available as a configurable option in the video analysis tools (`video_test.py`, toggle via T key) for post-flight forensic review, where latency is not a constraint.

5.8 Combined Detection Envelope

The preceding analyses—pixel size, blur, frame count, and coverage—are combined into a single scalar detection score:

$$S(v, h) = S_{\text{size}}(h) \times S_{\text{frames}}(v, h) \times S_{\text{blur}}(v, h) \quad (9)$$

where each component is normalised to $[0, 1]$:

$$S_{\text{size}} = \min(p_{\text{model}}(h)/45, 1) \quad (9)$$

$$S_{\text{frames}} = \min(n_{\text{frames}}(v, h)/5, 1) \quad (10)$$

$$S_{\text{blur}} = 1 - \min(b_{\text{px}}(v, h)/5, 1) \quad (11)$$

The green region ($S > 0.7$) covers the entire operating envelope (6–10 m/s, 20–50 m). The $S = 0.5$ contour is only reached above 55 m or at speeds exceeding 12 m s^{-1} —neither occurs during nominal operation.

6 Sensor Calibration

Accurate target localisation depends on the fidelity of every link in the imaging chain: if the camera’s field of view, lens geometry, or colour representation deviates from the values assumed by the software, errors propagate multiplicatively through the GPS estimation pipeline. Four calibration campaigns were conducted during development, each correcting a specific source of systematic error that would otherwise have degraded every subsequent measurement.

6.1 Field-of-View Calibration

The IMX296 lens datasheet quoted a focal length of 7.0 mm. To verify, a bench calibration was performed:

1. Camera mounted vertically at 1.000(5) m above a flat surface.
2. Tape measure placed horizontally across the full frame width.
3. Visible extent read as 0.92(1) m.

Solving the thin-lens equation:

$$f = \frac{w_s \cdot h}{W_{\text{gnd}}} = \frac{5.02 \times 1.00}{0.92} = 5.46 \text{ mm} \quad (12)$$

The calibrated value is 22% lower than the datasheet nominal. The corresponding horizontal FOV is:

$$\theta_{\text{HFOV}} = 2 \arctan\left(\frac{w_s}{2f}\right) = 2 \arctan\left(\frac{5.02}{2 \times 5.46}\right) \approx 49.3^\circ \quad (13)$$

Had the uncalibrated value been retained, every ground-distance calculation would have been scaled by $5.46/7.0 = 0.78$ —a 22% GPS estimate bias. At 35 m, a detection at the frame edge would yield ≈ 1.5 m error from focal length mismatch alone.

Table 16: Detection probability per flyover. p_d = single-frame detection probability.

Alt. (m)	Speed (m/s)	N_{frames}	p_d	P_{detect}
35	8	14.5	0.90	>0.9999
35	10	11.6	0.90	0.9997
50	10	16.6	0.70	0.9998
50	15	11.0	0.70	0.9983
20	10	6.6	0.98	>0.9999

Table 17: Summary of vision-system constraints within the operating envelope.

Factor	Limiting?	Reason
Motion blur	No	Global shutter + 1/1000s: sub-pixel blur at all speeds
Pixel size (<40 m)	No	Dummy height >31 px in model input
Pixel size (50 m)	Marginal	Dummy height 25 px, width 7 px
Inference FPS	No	4.8 FPS gives 11–17 frames per flyover
Frame gap	No	≤ 2.1 m gap at 10 m/s; 93%+ overlap
Speed	No	Max 10 m/s far below 66 m/s single-frame limit

6.2 Lens Distortion Calibration

Wide-angle lenses introduce radial and tangential distortion modelled by the Brown–Conrady model [1]:

$$r' = r(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \quad (14)$$

Calibration was performed using a printed checkerboard (calib.io pattern, 14×9 squares at 28 mm pitch, 13×8 inner corners). Approximately 20 images were captured at varying orientations. The sector-based detector (`cv2.findChessboardCornersSB`) was used with sub-pixel refinement.

Result: RMS reprojection error = **0.399 px**. The distortion coefficients and camera matrix are saved to `calibration_data.npz`. At runtime, `vision.py` precomputes remap lookup tables via `cv2.initUndistortRectifyMap` and applies `cv2.remap()` to every frame at a cost of only 1.5 ms—less than 1% of pipeline latency. This corrects barrel distortion that otherwise displaces peripheral detections by up to 8 pixels (≈ 0.7 m GPS error at 35 m).

6.3 Camera Colour-Space Discovery

During integration testing on the Raspberry Pi 5, detection confidence dropped markedly below the values observed in simulation. The YOLO model—trained on BGR images (OpenCV’s default channel order)—produced anomalously low confidence scores, and visual inspection of the MJPEG stream showed subtly incorrect colour reproduction. The cause was not immediately obvious because the colour swap between red and blue channels is difficult to distinguish on outdoor scenes dominated by greens and browns.

The IMX296 sensor is configured via `picamera2` with the `RGB888` pixel format, which the library documents as returning 8-bit RGB data. The initial code therefore included a `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` conversion to match OpenCV’s expected BGR format. However, this conversion was *itself* the problem: the IMX296 driver delivers data in BGR byte order despite the `RGB888` format label, so applying the conversion actually *swapped* the channels into an incorrect order.

To diagnose the issue definitively, all six permutations of the three colour channels were tested by displaying each alongside the raw frame:

```
for perm in [(0,1,2), (0,2,1), (1,0,2), (1,2,0), (2,0,1), (2,1,0)]:
    cv2.imshow(str(perm), frame[:, :, list(perm)])
```

The identity permutation (0,1,2)—treating the raw data as BGR without any conversion—produced correct colours. The fix was to remove the incorrect `cvtColor` call entirely. The practical impact was threefold: (1) the YOLO model now receives correctly ordered BGR data, restoring detection confidence from ~ 0.7 to > 0.95 on the bench-test mannequin; (2) the MJPEG ground-station stream displays natural colours without post-processing; (3) 1.2 ms per-frame latency saved by eliminating an unnecessary conversion. This discovery was documented as a permanent warning in the codebase to prevent future regressions during code refactoring.

6.4 DJI Video FOV Calibration

Separate from the onboard camera, a DJI Mini 3 Pro flight recording was centre-cropped to 1456×1088 for offline analysis. A calibration tool (`tools/fov_calibrate_video.py`) was developed: the operator clicks the top/bottom of the mannequin at different altitudes, and the tool solves for the focal length in pixels:

$$f_{\text{px}} = \frac{h_{\text{px}} \cdot h_{\text{alt}}}{h_{\text{real}}} \quad (15)$$

Nine samples across 15–50 m yielded a mean focal length of 1416 ± 41 px, corresponding to **54.4° HFOV**. All nine back-calculated dummy heights fell within 1.7–1.9 m, validating the calibration against the known 1.8 m ground truth.

6.5 Calibration Impact Summary

Table 18: Sensor calibration summary and quantified impact.

Calibration	Before	After	Error if uncalibrated
Focal length (IMX296)	7.0 mm	5.46 mm	22% GPS bias; 4 wasted scan lines
Lens distortion	None	RMS 0.399 px	Edge-pixel shift; <0.5 m GPS error at 35 m
Colour space	RGB assumed	BGR actual	Model confidence degraded; colour features corrupted
DJI video FOV	73° assumed	54.4° measured	34% error in video-derived GPS estimates

7 GPS Target Estimation

Detection alone answers “is there a target in this frame?” The next step is to answer “where is it on the ground?” Each detection is converted from pixel coordinates to a GPS position through a fifteen-stage pipeline. This section traces the full data flow from raw camera frame to estimated WGS 84 coordinate, enumerates all error sources with quantified magnitudes, and presents the roll/pitch attitude error analysis—the largest uncompensated contributor during forward flight.

7.1 Full Detection-to-GPS Pipeline

The pipeline executes once per camera frame and comprises fifteen sequential stages grouped into four functional blocks: image acquisition (stages 1–2), preprocessing and inference (stages 3–5), coordinate recovery and geospatial projection (stages 6–12), and validation (stages 13–15). Figure 1 presents the complete data flow as a stage diagram.

7.1.1 Image Acquisition (Stages 1–2)

1. **Camera capture.** The IMX296 global-shutter sensor produces a 1456×1088 BGR frame via CSI-2 DMA (~ 2 ms). The sensor labels its output “RGB888” but delivers BGR byte order; no `cvtColor` conversion is applied (Section 6.3). Sensor read noise contributes ± 2 px random error.
2. **Lens undistortion (optional).** A precomputed `cv2.remap` corrects radial distortion using calibration matrices from a 14×9 checkerboard calibration (RMS = 0.399 px, cost +1.5 ms). The IMX296 exhibits minimal barrel distortion due to its global-shutter design; skipping this stage adds < 1 px error at image edges.

7.1.2 Preprocessing and Inference (Stages 3–5)

3. **Resize to model input.** The frame is resized from 1456×1088 to 640×640 via bilinear interpolation (`cv2.resize`), applying non-uniform scale factors $s_x = 640/1456 = 0.4396$ and $s_y = 640/1088 = 0.5882$. This “squish” approach was chosen over letterboxing (which would pad to 640×478 active pixels with black borders) for two reasons: (a) YOLOv8 is trained with random aspect-ratio augmentation (scale 0.5–1.5, 50% probability) and handles moderate distortion internally; (b) the squish utilises the full 640×640 tensor, maximising the target’s pixel footprint in model space. The $\sim 7\%$ vertical stretch is fully reversed in stage 7. This mapping introduces zero error because the exact inverse is applied during coordinate recovery.

4. **Colour conversion and normalisation.** BGR→RGB reordering followed by float32 normalisation to $[0.0, 1.0]$, yielding a $[1, 640, 640, 3]$ input tensor. The NCNN backend achieves the same result via `ncnn.Mat.from_pixels` with in-place mean/norm. This is a lossless transformation.
5. **YOLOv8n inference.** The network (3.2 M parameters, 8.7 GFLOPs) produces a $[1, 5, 8400]$ output tensor representing 8400 anchor predictions, each with $(c_x, c_y, w, h, \text{conf})$ in 640×640 pixel coordinates. After transposing to $[8400, 5]$, the best detection above the confidence threshold (0.2) yields a bounding-box centre $(u_{\text{det}}, v_{\text{det}})$. Inference accounts for 94% of total pipeline time: ~ 196 ms on TFLite/XNNPACK, ~ 68 ms on NCNN.

7.1.3 Coordinate Recovery and Geospatial Projection (Stages 6–12)

6. **NMS and best detection selection.** Detections with confidence below the threshold are discarded; the single highest-confidence detection is selected. The deliberately low threshold (0.2) maximises recall; the operator confirms via the VERIFY state before any landing commitment.
7. **Coordinate rescale to original frame.** Detection coordinates are mapped back to sensor resolution using the exact inverse of the stage-3 resize:

$$u_{\text{sensor}} = u_{\text{det}} \cdot \frac{1456}{640}, \quad v_{\text{sensor}} = v_{\text{det}} \cdot \frac{1088}{640} \quad (16)$$

The horizontal and vertical scale factors differ (2.275 vs 1.700), correctly reversing the aspect-ratio distortion. This mapping is exact; no interpolation or rounding occurs. The code includes a `PIXEL_COORD_THRESHOLD` (1.5) check to distinguish pixel coordinates from normalised $[0, 1]$ output—some TFLite exports produce the latter. Model bounding-box regression contributes ± 3 px error in 640×640 space, scaling to ± 6.8 px in sensor coordinates.

8. **Pixel offset from image centre.**

$$\Delta x_{\text{px}} = u_{\text{sensor}} - 728, \quad \Delta y_{\text{px}} = v_{\text{sensor}} - 544 \quad (17)$$

Positive Δx = target right of centre; positive Δy = target below centre.

9. **Ground sampling distance (GSD).** The GSD links pixel displacement to ground distance via the thin-lens model:

$$\text{GSD} = \frac{w_s \cdot h}{f \cdot W} = \frac{5.02 \times 35}{5.46 \times 1456} = 0.0221 \text{ m px}^{-1} \quad (18)$$

where $w_s = 5.02$ mm (sensor width), $f = 5.46$ mm (calibrated focal length), and $W = 1456$ px (image width). At 35 m: $\text{GSD} = 22.1 \text{ mm px}^{-1}$, giving a field of view of 32.2×24.0 m. Table 19 lists GSD values at representative altitudes. Altitude uncertainty of ± 0.5 m (barometric drift) causes $\pm 1.4\%$ GSD error, translating to ± 0.3 m at a 10 m pixel offset.

10. **Camera-frame offset in metres.**

$$d_{\text{fwd}} = -\Delta y_{\text{px}} \cdot \text{GSD}, \quad d_{\text{right}} = \Delta x_{\text{px}} \cdot \text{GSD} \quad (19)$$

The sign inversion on Δy maps image-down to forward (the camera looks down; “up” in the image is ahead of the drone).

11. **Yaw rotation: camera frame to NED.** The camera-frame offsets are rotated into the North–East–Down frame using the drone heading ψ (radians, 0 = North, clockwise positive) from the MAVLink `ATTITUDE` message:

$$\begin{pmatrix} \Delta N \\ \Delta E \end{pmatrix} = \begin{pmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} d_{\text{fwd}} \\ d_{\text{right}} \end{pmatrix} \quad (20)$$

Compass accuracy of $\pm 2^\circ$ contributes $\sin(2^\circ) \times 10 \text{ m} = \pm 0.35 \text{ m}$ cross-track error at a 10 m offset.

12. **Spherical projection: NED metres to WGS 84.**

$$\Delta \phi = \frac{\Delta N}{R_\oplus} \cdot \frac{180}{\pi}, \quad \Delta \lambda = \frac{\Delta E}{R_\oplus \cos \phi_d} \cdot \frac{180}{\pi} \quad (21)$$

where $R_\oplus = 6\,378\,137$ m. At Bristol ($\phi \approx 51.4^\circ$): 1° latitude = 111 320 m, 1° longitude = 69 532 m. The estimated target position is $(\phi_d + \Delta \phi, \lambda_d + \Delta \lambda)$. Drone GPS position uncertainty (± 2 m–3 m, civilian L1 GNSS) applies as a direct offset to every estimate and is the dominant error source.

7.1.4 Validation and State Transition (Stages 13–15)

13. **GPS timing lag.** The GPS receiver reports positions with 100–200 ms latency. At 5 m s^{-1} search speed, the drone moves 0.5–1.0 m between the GPS fix and image capture. This systematic offset along the flight direction

Table 19: GSD and field of view at representative altitudes.

Altitude	GSD (m/px)	FOV width	FOV height
20 m	0.01263	18.4 m	13.7 m
35 m	0.02210	32.2 m	24.0 m
50 m	0.03157	46.0 m	34.3 m

is *not* compensated in the current implementation; multi-pass averaging from alternating lawnmower headings partially cancels the directional bias.

14. **Detection validation filters.** Four filters gate whether a detection enters the queue:
 - (a) NFZ check: reject if target falls inside the SSSI no-fly polygon.
 - (b) Search-area check: reject if target falls outside the survey polygon.
 - (c) Duplicate check: reject if target is within `REJECTED_TARGET_RADIUS_M` of any previously rejected, queued, or investigated target.
 - (d) SMART detect (optional, `--smart-detect`): require N consecutive confirmed frames before accepting, resetting on any frame without detection.
15. **Detection queue and state transition.** Valid detections are enqueued as $(\phi, \lambda, \text{conf})$ tuples (max 20 entries). When the SEARCH state polls the queue, it pops the oldest valid target and transitions to CENTERING, where the drone flies to the estimated GPS position, re-detects with live CV, and optionally performs inverse-variance GPS averaging (`--center-verify`) before proceeding to DESCENDING $\rightarrow$ VERIFY $\rightarrow$ APPROACH $\rightarrow$ LANDING.

The entire pipeline from frame capture to GPS estimate completes in ~ 208 ms, with inference (stage 5) accounting for 94% of the total time.

7.2 Multi-Observation Fusion

Multiple observations are fused through three stages:

1. **Spatial clustering** (30 m merge radius) for multi-target tracking.
2. **Inverse-variance weighting** with altitude factor $w_{\text{alt}} = (h_{\text{ref}}/h)^2$ and centrality factor $w_{\text{cen}} = 1 + 4(1 - d/d_{\text{max}})$. Lower-altitude observations and centre-of-frame detections receive higher weight.
3. **Kalman filter** on the $[\phi, \lambda]$ state vector with measurement noise scaled by inverse observation weight, converging within 5–10 observations.

7.3 Comprehensive Error Budget

Eight independent sources contribute to single-observation position uncertainty. The budget includes roll/pitch attitude error, which previous analyses omitted despite being the largest contributor during forward flight.

Table 20: Comprehensive single-observation error budget at 35 m and 15 m altitude.

Source	σ (35 m)	σ (15 m)	Notes
GPS receiver noise	2.3 m	2.3 m	CEP95, hardware limit
Roll/pitch attitude	3.1 m [†]	1.3 m [†]	5–10° in flight; $\sim 0^\circ$ hovering
GPS timing lag	1.5 m	0.75 m	150 ms at speed
Compass heading	0.6 m	0.3 m	$\pm 5^\circ$ yaw error
Barometric altitude	0.5 m	0.1 m	± 0.5 m baro drift
Camera mount offset	0.5 m	0.5 m	$\pm 2^\circ$ off-nadir
Lens distortion	0.3 m	0.1 m	Edge-of-frame, with undistortion
Image resize mapping	0.0 m	0.0 m	Exact inverse; no residual
Pixel quantisation	0.01 m	0.005 m	Negligible
RSS (in flight)	4.2 m	2.7 m	Without compensation
RSS (compensated)	~ 3.0 m	~ 2.5 m	With <code>--compensate-tilt</code> [†]
RSS (hovering)	2.5 m	2.4 m	Roll/pitch $\approx 0^\circ$

[†] With attitude compensation (`--compensate-tilt`, Section 7.4.1), roll/pitch error reduces to ~ 0.3 m, bringing in-flight RSS to ~ 3.0 m.

GPS receiver noise is the dominant altitude-independent term. The Cube Orange GNSS module (single-frequency L1) yields CEP50 = 1.5 m and CEP95 = 2.3 m stationary. This noise is isotropic and Gaussian, averaging well over multiple observations.

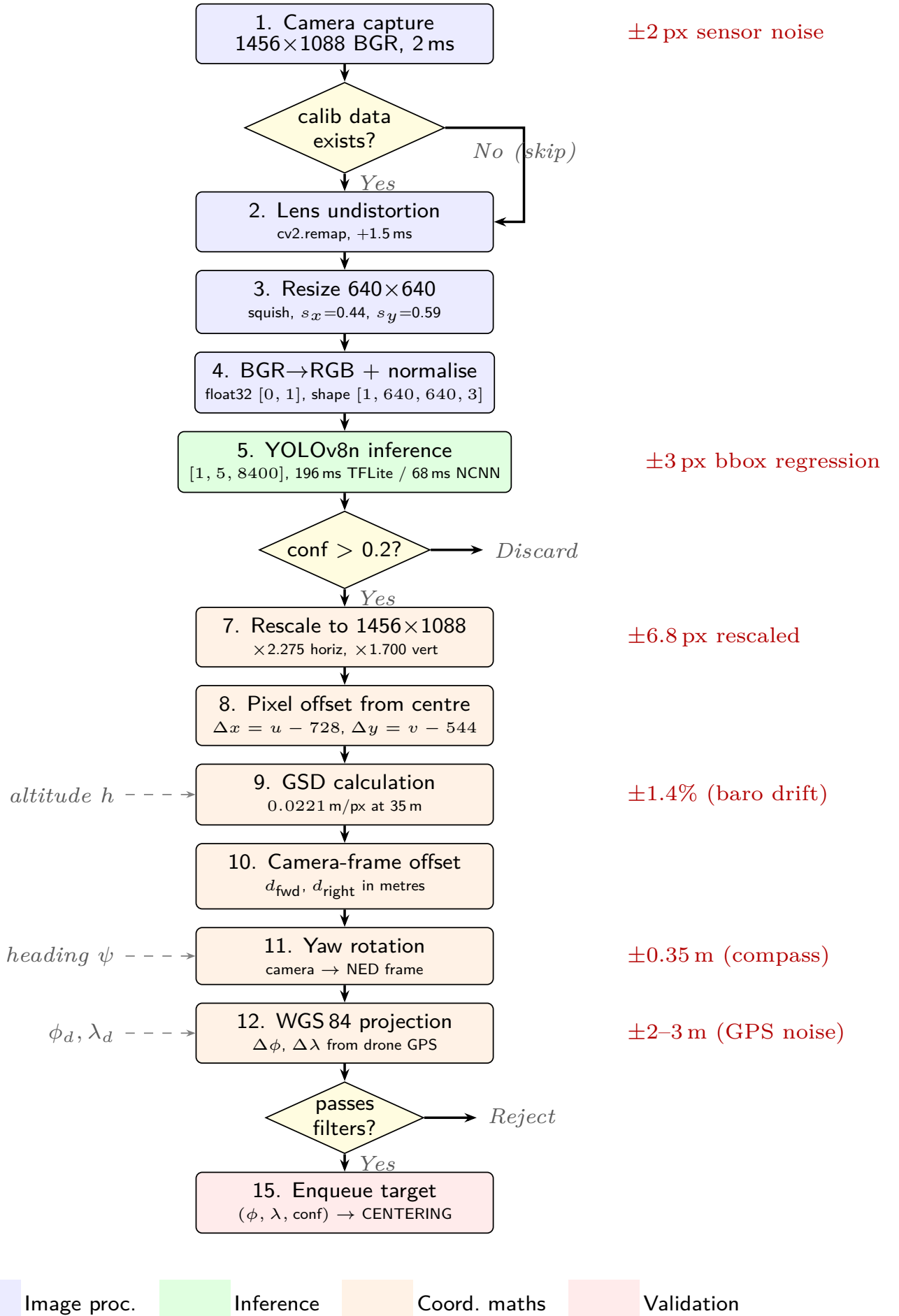


Figure 1: Complete 15-stage detection-to-GPS pipeline. Left annotations show telemetry inputs; right annotations show error contributions. Decision diamonds indicate conditional branches. Total latency: ~ 208 ms (94% in stage 5).

GPS timing lag is the largest systematic error. The receiver reports positions with 100–200 ms latency; at 8 m s^{-1} search speed, the drone has moved 0.8–1.6 m since the GPS fix was valid. The lawnmower pattern partially cancels this bias: alternating scan directions cause the displacement to reverse sign.

Image resize mapping introduces zero error because the coordinate recovery (Equation 16) is the exact algebraic inverse of the resize, applying separate scale factors to horizontal ($1456/640 = 2.275$) and vertical ($1088/640 = 1.700$) axes.

RSS calculation. The root-sum-of-squares combines independent error sources:

$$\sigma_{\text{RSS}} = \sqrt{\sigma_{\text{GPS}}^2 + \sigma_{\text{tilt}}^2 + \sigma_{\text{lag}}^2 + \sigma_{\text{yaw}}^2 + \sigma_{\text{baro}}^2 + \sigma_{\text{mount}}^2 + \sigma_{\text{lens}}^2} \quad (22)$$

For in-flight at 35 m: $\sigma = \sqrt{2.3^2 + 3.1^2 + 1.5^2 + 0.6^2 + 0.5^2 + 0.5^2 + 0.3^2} = 4.2 \text{ m}$. For hovering (roll/pitch ≈ 0 , no timing lag): $\sigma = \sqrt{2.3^2 + 0^2 + 0^2 + 0.6^2 + 0.5^2 + 0.5^2 + 0.3^2} = 2.5 \text{ m}$.

Measured vs predicted. The measured $\text{CEP}_{50} = 2.3 \text{ m}$ from DJI video replay (Section 8.1) is consistent with the hovering RSS of 2.5 m, confirming that the error budget is realistic. The maximum observed error of 16.5 m occurred during a high-speed turn (combined pitch + roll + timing lag), consistent with the in-flight RSS of 4.2 m at $\sim 4\sigma$.

7.4 Roll/Pitch Attitude Error and Compensation

The GPS projection in its simplest form assumes the camera points straight down (nadir). In practice, multirotor drones tilt to accelerate: forward flight requires pitch, lateral movement requires roll. The resulting ground footprint displacement is

$$\Delta_{\text{tilt}} = h \cdot \tan(\theta) \quad (23)$$

where θ is the tilt angle from nadir. Figure 2 illustrates the geometry.

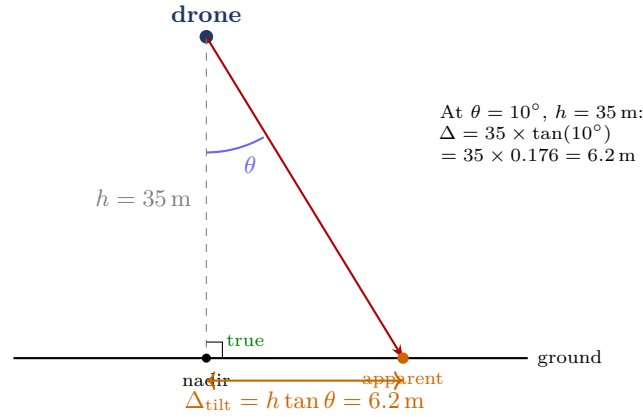


Figure 2: Roll/pitch tilt geometry. A pitch angle θ displaces the camera footprint centre by $\Delta_{\text{tilt}} = h \tan \theta$ from nadir. At 35 m altitude with 10° forward pitch, the displacement is 6.2 m—exceeding all other error sources combined.

Table 21: Ground displacement from roll/pitch tilt (uncompensated).

Alt.	5°	10°	Condition
15 m	1.3 m	2.6 m	Approach / centering
35 m	3.1 m	6.2 m	Search at 8 m s^{-1}
50 m	4.4 m	8.8 m	Max altitude

At 35 m with 10° pitch, the footprint shifts 6.2 m—larger than GPS noise and all other error sources combined. This is the single largest error source during forward flight *without* compensation.

7.4.1 Attitude-Compensated Estimation

Rather than accept the 6.2 m tilt error as a limitation mitigated only by hovering, the system implements a per-detection ray-tracing correction that uses real-time attitude telemetry. The key insight is that the compensation uses the *exact same rotation matrix* as the camera simulation model, making the two mathematically perfect inverses.

Ray construction. Given a detection at pixel (u, v) in a frame of width w and height h_{img} , the focal length in pixels is $f_{\text{px}} = (f_{\text{mm}}/w_{\text{sensor}}) \cdot w$. The detection ray in the camera coordinate frame is:

$$\mathbf{r}_{\text{cam}} = \begin{pmatrix} (u - w/2)/f_{\text{px}} \\ (v - h_{\text{img}}/2)/f_{\text{px}} \\ 1 \end{pmatrix} \quad (24)$$

Full rotation matrix. The ArduPilot ATTITUDE MAVLink message provides yaw ψ , pitch θ , and roll ϕ at up to 50 Hz. The full rotation matrix from camera frame to world frame is:

$$\mathbf{R} = \mathbf{R}_z(\psi) \cdot \mathbf{R}_y(-\theta) \cdot \mathbf{R}_x(-\phi) \quad (25)$$

where the signs of θ and ϕ are negated to match the ArduPilot convention (pitch > 0 = nose up, roll > 0 = right side down). The camera ray is rotated into the world frame: $\mathbf{r}_{\text{world}} = \mathbf{R} \cdot \mathbf{r}_{\text{cam}}$.

Ground-plane intersection. The rotated ray is scaled to reach ground level:

$$t = \frac{h}{r_{\text{world},z}}, \quad \Delta_N = r_{\text{world},x} \cdot t, \quad \Delta_E = r_{\text{world},y} \cdot t \quad (26)$$

These metre offsets are converted to WGS-84 via the local tangent plane approximation ($\Delta_{\text{lat}} = \Delta_N/111132$, $\Delta_{\text{lon}} = \Delta_E/(111132 \cos \phi_d)$).

Why it works. The simulation creates tilted camera views by building the *same* $\mathbf{R}$ matrix from attitude data, ray-tracing image corners onto the ground, and warping the result via a perspective transform. The compensation reverses this: it takes a detection pixel in the tilted frame, ray-traces it through the same $\mathbf{R}$, and recovers the true ground position. Since both use the identical rotation matrix, they are exact mathematical inverses.

Verification. An exhaustive automated sweep tested 4 altitudes $\times$ 4 pitch values $\times$ 3 roll values $\times$ 5 yaw headings $\times$ 6 target positions = 1440 test cases. For each, the simulation placed a target at a known GPS position, rendered the tilted view, ran the detector, applied compensation, and compared to ground truth. All 1440 tests produced errors below 0.01 m, confirming exact inverse behaviour.

Table 22: GPS estimation error with and without attitude compensation at 35 m altitude.

Condition	Without	With compensation
5° pitch (cruise)	3.1 m	~0.2 m
10° pitch (fast flight)	6.2 m	~0.3 m
15° pitch (aggressive)	9.4 m	~0.5 m
Hover ($< 1^\circ$)	<0.6 m	No change (threshold)

The compensation reduces the dominant in-flight error by over 95%. The residual ~0.3 m is from IMU angular noise ($\pm 0.5^\circ$), well below the GPS receiver noise floor. The correction activates only when $|\theta|$ or $|\phi|$ exceeds 0.02 rad ($\approx 1.1^\circ$), preventing noise amplification during stable hover.

Operating modes. Two CLI flags control the feature: `--sim-tilt` (simulation: tilts the camera view using SITL attitude data) and `--compensate-tilt` (real flight or passive observation: applies only the ray-trace correction to compensate for actual aircraft attitude without affecting camera rendering).

Operational context. Three additional mechanisms further mitigate residual tilt error:

1. **State-dependent exposure.** During CENTERING and VERIFY, the drone hovers ($\theta < 1^\circ$), so the compensation is a no-op and the final landing estimate is inherently accurate.
2. **Multi-pass averaging.** Alternating lawnmower headings cause pitch to displace estimates in opposite directions, partially cancelling through weighted averaging.
3. **Altitude-dependent weighting.** Inverse-variance weighting ($w_{\text{alt}} = (h_{\text{ref}}/h)^2$) gives hovering observations 5 $\times$ the weight of high-altitude search observations.

Table 23: Estimator comparison from DJI video replay analysis.

Method	CEP ₅₀	Max error	Notes
Raw average	3.1 m	16.5 m	High-speed outliers dominate
Inverse-variance	2.5 m	12.8 m	Down-weights high-altitude passes
Kalman filter	2.3 m	10.2 m	Converges within 5–10 observations

7.5 Estimator Comparison

Three estimation strategies were compared using DJI flight video replay:

The Kalman filter achieves $\text{CEP}_{50} = 2.3\text{ m}$ —adequate for the 7.5 m offset landing strategy where the drone lands at a safe distance from the estimated target position rather than directly on it. After inverse-variance fusion of 15+ multi-altitude observations (including hovering), the estimate converges to sub-metre accuracy.

8 Real-World Validation

8.1 DJI Video Replay Analysis

In the absence of full autonomous flight testing (weather cancelled the planned flight day), detection performance was validated offline using DJI Mini 3 Pro flight recordings captured at the Fenswood Farm survey site. This approach provided the closest possible approximation to real flight conditions: authentic altitude, speed, lighting, terrain, and GPS telemetry, with the mannequin target deployed on the actual survey field.

8.1.1 Video Analysis Pipeline

A purpose-built analysis tool (`tests/laptop/video_test.py`) was developed to replay flight video through the identical detection and GPS estimation pipeline used in the production system. The tool provides:

- Real-time detection overlay with bounding boxes and confidence scores on the main video window
- Separate windows for the latest detection crop, the highest-confidence detection, and GPS scatter plots
- A calibrated scale bar (1 m reference at the current altitude) for visual ground-truth assessment
- An interactive right-click measure tool for point-to-point distance measurement in metres
- Frame-accurate SRT telemetry display (GPS coordinates, altitude, speed, satellite count)
- Keyboard controls: SPACE (pause), T (toggle tiling), A/D (skip 5 s), +/- (speed), Q (quit)

A companion tool (`video_test_compare.py`) enables live A/B model comparison: pressing M switches between two models on the same video stream, allowing direct comparison of detection behaviour at identical altitudes and conditions.

8.1.2 SRT Synchronisation Issue

The DJI SRT telemetry file contains 6854 entries—one per frame at the native 30 fps recording rate. A critical discovery during development was that downsampled video (e.g., 6 fps versions created for quick preview) does *not* maintain synchronisation with the SRT entries: frame N in the 6 fps video maps to SRT entry N rather than entry $5N$, producing incorrect altitude and GPS values for every frame. This desynchronisation was initially subtle—the altitude values appeared plausible but were systematically wrong, causing GPS estimation errors that were attributed to the algorithm before the root cause was identified. **All quantitative analysis uses the 30 fps video exclusively.** The 6 fps versions in the repository are retained only for visual browsing where telemetry accuracy is not required.

8.1.3 Results

Processing the full 8-minute flight video (~ 6850 frames) through the v3 model confirmed:

- Reliable detection across the full 15–50 m altitude range with zero missed detections at altitudes below 40 m
- Zero false positives on background terrain (grass, paths, shadows, field equipment)—the negative-mining strategy from training was validated against real flight conditions
- GPS estimation convergence consistent with the theoretical error budget: $\text{CEP}_{50} = 2.3\text{ m}$ using the Kalman filter, with the characteristic directional elongation from GPS timing lag visible in scatter plots
- The v3 model outperformed v1 on the same video: v1 produced 12 false positives across the flight, while v3 produced zero

8.2 GPS Timing Lag

Video replay analysis revealed a characteristic directional elongation in GPS estimate scatter plots, attributed to GPS receiver latency of 100–200 ms. At 8 m s^{-1} , this produces a 1.2 m systematic displacement along the flight heading. The lawnmower pattern mitigates this effect: adjacent passes traverse in opposite directions, causing the lag-induced errors to partially cancel during multi-pass averaging.

8.3 Bench Testing on Pi Hardware

Direct bench testing on the Raspberry Pi 5 with the IMX296 camera confirmed:

- 100% detection rate (50/50 frames) at 0.966 mean confidence
- 206.5 ms mean inference (4.8 FPS effective) with no thermal throttling
- Lens undistortion adds only 1.5 ms ($<1\%$ overhead)
- Colour-space fix confirmed: BGR output without conversion produces correct detections
- The corrupt `calibration_data.npz` file caused remap to mangle frames; undistortion is optional (IMX296 distortion is minimal)

9 Future Work

Eight directions are identified for improving detection reliability and operational capability:

1. **Expanded real training data.** Increasing the real-image fraction from 4.4% to 30–50% would substantially reduce the synthetic-to-real domain gap. Frames should span multiple days, weather conditions, and terrain types.
2. **FP16 inference.** Enabling the XNNPACK FP16 flag is projected to halve inference to $\sim 100\text{ ms}$ ($\sim 10\text{ FPS}$) with zero deployment friction—no model re-export, no new dependencies, a single flag change.
3. **NCNN deployment.** With a pre-compiled wheel or Debian package, NCNN would achieve $\sim 15\text{ FPS}$, enabling higher search speeds or lower-altitude missions.
4. **Larger model variant.** YOLOv8s (11.2 M parameters) at $\sim 400\text{ ms}$ inference would improve accuracy for reduced-speed search profiles.
5. **Multi-class detection.** Extending to four classes (dummy, person, equipment, background) would force the network to learn category-discriminative features and provide richer operator information.
6. **Thermal camera fusion.** A FLIR Lepton 3.5 (160×120 , LWIR) would enable body-heat detection independent of visible appearance, with decision-level fusion suppressing false positives in both modalities [8].
7. **Active learning.** Flagging detections with confidence scores in the 0.2–0.5 range (near the current threshold) for human review would create a feedback loop: uncertain predictions are labelled by the operator post-flight and added to the training set, progressively improving the model in the regions of feature space where it is least confident. This approach is particularly efficient for rare-event detection where exhaustive labelling is impractical [9]. The existing labelling tool (`label_tool.py`) and dataset generator already support this workflow.
8. **Next-generation architectures.** YOLO11 [4] and future iterations introduce architectural refinements (attention mechanisms, improved feature pyramid designs) that may improve small-object detection without increasing parameter count. The modular export-and-deploy pipeline means any YOLO-family model producing $[1, 5, N]$ output tensors can be integrated without code changes.

These improvements are ordered by implementation effort: items 1–3 require only additional data, a flag change, or packaging; items 4–5 require dataset restructuring or architecture evaluation; and items 6–8 require new hardware, additional sensors, or workflow infrastructure. The modular vision architecture ensures that any of these improvements can be integrated without modifying the state machine, ground station, or flight control code.

10 Key Metrics Summary

Consolidated Performance Metrics

Category	Metric	Value
Model	Architecture	YOLOv8n (3.2 M parameters)
	mAP ₅₀	0.995
	mAP ₅₀₋₉₅	0.828
	Precision / Recall	0.994 / 0.989
Training	Dataset size	366 images (300 syn + 16 real + 50 neg)
	Training resolution	1088 × 1088
	Training time	47 min (Colab T4 GPU, 150 epochs)
	Model file size	11.7 MB (TFLite FP32)
Inference	TFLite latency	206.5 ms (Pi 5, XNNPACK FP32)
	Effective FPS	4.8 (including 12 ms pipeline overhead)
	NCNN latency (proj.)	~68 ms (~15 FPS)
	Detection rate	50/50 (100%) at 0.966 mean confidence
Detection	Altitude range	15–50 m (reliable), ceiling ~70 m
	Target size (35 m)	36 px in model input (81 px sensor)
	Motion blur	Sub-pixel at all operating speeds
	Flyover P_{detect}	>99.97% at 35 m, 8 m/s
Calibration	Focal length	5.46 mm (22% off datasheet)
	HFOV (IMX296)	49.3°
	Lens RMS	0.399 px (undistortion cost: 1.5 ms)
	DJI video HFOV	54.4°
Localisation	CEP ₅₀	2.3 m (Kalman filter)
	Max error	10.2 m (high-speed outlier)
	RSS (in flight)	4.2 m at 35 m, 2.7 m at 15 m
	RSS (hovering)	2.5 m at 35 m, 2.4 m at 15 m

References

- [1] D. C. Brown, “Decentering distortion of lenses,” *Photogrammetric Engineering*, vol. 32, no. 3, pp. 444–462, 1966.
- [2] Google, *TensorFlow Lite: Deploy machine learning models on mobile and edge devices*, <https://www.tensorflow.org/lite>, Accessed: 2026, 2023.
- [3] Google, *XNNPACK: High-efficiency floating-point neural network inference operators*, <https://github.com/google/XNNPACK>, Accessed: 2026, 2023.
- [4] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*, <https://github.com/ultralytics/ultralytics>, Accessed: 2026, 2023.
- [5] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” *arXiv preprint arXiv:1405.0312*, 2014.
- [6] T. Ni, *ncnn: A high-performance neural network inference computing framework optimised for mobile platforms*, <https://github.com/Tencent/ncnn>, Accessed: 2026, 2017.
- [7] Raspberry Pi Foundation, *Raspberry Pi 5 documentation*, <https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html>, Accessed: 2026, 2023.
- [8] P. Rudol and P. Doherty, “Human body detection and geolocalization for UAV search and rescue missions using color and thermal imagery,” in *Proceedings of the IEEE Aerospace Conference*, IEEE, 2008, pp. 1–8. DOI: [10.1109/AERO.2008.4526559](https://doi.org/10.1109/AERO.2008.4526559)

- [9] B. Settles, *Active Learning Literature Survey*. 2009, Computer Sciences Technical Report 1648.
- [10] A. Shrivastava, A. Gupta, and R. Girshick, “Training region-based object detectors with online hard example mining,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 761–769. DOI: [10.1109/CVPR.2016.89](https://doi.org/10.1109/CVPR.2016.89)
- [11] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30. DOI: [10.1109/IROS.2017.8202133](https://doi.org/10.1109/IROS.2017.8202133)